100+ Important React.js Interview questions

Author: Sandeep Pal

React.js Interview Questions

Basics & Core Concepts

1. What is React and why would you use it?

React is a **JavaScript library** for building **user interfaces**, mainly for **single-page applications**. It helps developers create **reusable UI components**, manage state efficiently, and update the UI in a performant way.

✅ **Why use React?**

- Easy to break UIs into components
- Efficient updates with Virtual DOM
- Rich ecosystem (React Router, Redux, etc.)
- Backed by Facebook and a large community

https://www.linkedin.com/in/sandeeppal

---

2. What are the main features of React?

- **JSX** – JavaScript + XML
- **Components** – Reusable and composable
- **Virtual DOM** – Efficient DOM updates
- **Unidirectional data flow** – One-way data binding
- **Lifecycle methods (in class components)**
- **Hooks (in functional components)**

---

3. What is JSX? Can browsers read JSX directly?

JSX stands for **JavaScript XML**. It lets you write HTML-like syntax in JavaScript.

```
const element = <h1>Hello, world!</h1>;
```

🚫 **No, browsers cannot read JSX directly**. JSX needs to be **transpiled** (e.g., using Babel) into regular JavaScript.

---

4. How does React differ from other JavaScript frameworks?

Feature	React	Angular	Vue
Type	Library (UI focused)	Framework (full-featured)	Framework (lightweight)
DOM Handling	Virtual DOM	Real DOM	Virtual DOM
Data Binding	One-way	Two-way	Both supported

https://www.linkedin.com/in/sandeeppal

Learning Curve	Medium	Steep	Easy

---

5. What are components in React?

Components are the **building blocks** of a React application. Each component is an isolated, reusable piece of the UI.

✅ Example:

```
function Welcome(props) {
  return <h1>Hello, {props.name}</h1>;
}
```

---

6. Explain the difference between Class and Functional components.

Feature	Class Component	Functional Component
Syntax	ES6 Class	Function
State	`this.state`	`useState` hook
Lifecycle methods	Yes	Use `useEffect`, etc.
Code size	More verbose	Cleaner and shorter

✅ Example:

Class:

```
class Welcome extends React.Component {
  render() {
    return <h1>Hello, {this.props.name}</h1>;
  }
}
```

Functional:

https://www.linkedin.com/in/sandeeppal

```
function Welcome(props) {
  return <h1>Hello, {props.name}</h1>;
}
```

---

7. What is the Virtual DOM and how does React use it?

The **Virtual DOM** is a **lightweight in-memory representation** of the real DOM. React updates the Virtual DOM first and compares it with the previous version, only applying the necessary changes to the real DOM.

✅ Benefit: **Improves performance** by reducing real DOM operations.

---

8. How does React's reconciliation algorithm work?

React's **reconciliation algorithm** compares the new Virtual DOM with the old one using a **diffing algorithm**. It:

- Identifies what changed (nodes, attributes, etc.)
- Applies **minimum changes** to the actual DOM
- Uses **keys** to track list item changes efficiently

---

9. What are props in React?

Props (short for "properties") are **read-only data** passed from a parent component to a child.

✅ Example:

```
function Greeting(props) {
  return <h1>Hello, {props.name}!</h1>;
}

<Greeting name="Alice" />
```


https://www.linkedin.com/in/sandeeppal

---

10. What is state in React?

State is data that is **local to a component** and can change over time.

✅ Example using `useState`:

```
import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);
  return <button onClick={() => setCount(count +
1)}>{count}</button>;
}
```

---

11. How do you update state in React?

- **Functional component:** Use `setState` from `useState`.
- **Class component:** Use `this.setState()`.

✅ Example:

```
const [name, setName] = useState("John");
setName("Doe");
```

---

12. What is the difference between state and props?

Feature	Props	State
Mutable?	❌ No (read-only)	✅ Yes
Who sets it?	Parent component	Component itself

https://www.linkedin.com/in/sandeeppal

Used for	Passing data	Handling internal data

---

13. What is the purpose of keys in React lists?

Keys help React identify which items have changed, been added, or removed.

✅ Example:

```
{items.map(item => (
  <li key={item.id}>{item.name}</li>
))}
```

🛑 Avoid using array index as key unless absolutely necessary.

---

14. How do you conditionally render elements in React?

✅ Using ternary:

```
{isLoggedIn ? <Logout /> : <Login />}
```

✅ Using short-circuit:

```
{isVisible && <Sidebar />}
```

---

15. What are fragments in React and why are they useful?

Fragments let you return multiple elements without adding an extra DOM node.

✅ Example:

```
<>
  <h1>Title</h1>
  <p>Description</p>
</>
```

https://www.linkedin.com/in/sandeeppal

🔍 Equivalent to `<React.Fragment>` but shorter.

---

16. How does React handle events?

React uses **camelCase syntax** and passes functions directly.

✅ Example:

```
<button onClick={handleClick}>Click Me</button>
```

---

17. What are synthetic events in React?

React wraps native browser events in a **SyntheticEvent** object for **cross-browser compatibility**.

✅ Example:

```
function handleClick(e) {
  console.log(e); // SyntheticEvent
}
```

---

18. What is the difference between controlled and uncontrolled components?

Feature	Controlled	Uncontrolled
Input value managed by	React state	DOM itself
Access value via	`useState`	`ref`
Example use case	Forms with validation	Quick, simple input fields

✅ Controlled:

```
<input value={value} onChange={e => setValue(e.target.value)} />
```

https://www.linkedin.com/in/sandeeppal

✅ Uncontrolled:

```
<input ref={inputRef} />
```

---

19. How do you handle forms in React?

Use controlled components and `onChange` handlers.

✅ Example:

```
function Form() {
  const [name, setName] = useState("");
  const handleSubmit = e => {
    e.preventDefault();
    console.log(name);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input value={name} onChange={e => setName(e.target.value)} />
      <button type="submit">Submit</button>
    </form>
  );
}
```

---

20. What is lifting state up in React?

Lifting state up means moving state to the **nearest common ancestor** when multiple components need to share or modify it.

✅ Example:

```
function Parent() {
  const [value, setValue] = useState("");

  return (
```


https://www.linkedin.com/in/sandeeppal

```
    <>
      <Input value={value} onChange={setValue} />
      <Display value={value} />
    </>
  );
}

function Input({ value, onChange }) {
  return <input value={value} onChange={e =>
onChange(e.target.value)} />;
}

function Display({ value }) {
  return <p>{value}</p>;
}
```

https://www.linkedin.com/in/sandeeppal

React Hooks

21. What are React Hooks?

Hooks are functions that let you **"hook into" React state and lifecycle features** in **functional components**.

Before hooks, only **class components** could use state and lifecycle methods.

✅ Hooks introduced in React **16.8**.

---

22. Explain the `useState` hook with an example.

`useState` lets you add **state** to functional components.

✅ **Syntax:**

```
const [state, setState] = useState(initialValue);
```

✅ **Example:**

```
import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <button onClick={() => setCount(count + 1)}>
      Count: {count}
    </button>
  );
}
```

---

23. What is `useEffect` and when do you use it?

`useEffect` lets you **run side effects** in functional components (like data fetching, subscriptions, etc.).

✅ **Syntax:**

```
useEffect(() => {
  // Side effect
  return () => {
    // Cleanup
  };
}, [dependencies]);
```

✅ **Common use cases:**

- Fetching data

https://www.linkedin.com/in/sandeeppal

- Event listeners
- Updating DOM directly
- Subscribing to services

---

24. How do you mimic `componentDidMount` and `componentWillUnmount` with hooks?

✅ **componentDidMount:**

```
useEffect(() => {
  console.log("Component mounted");
}, []); // Empty array = run once on mount
```

✅ **componentWillUnmount:**

```
useEffect(() => {
  const id = setInterval(() => console.log("tick"), 1000);

  return () => {
    clearInterval(id); // Cleanup
    console.log("Component unmounted");
  };
}, []);
```

---

25. What is the difference between `useEffect` and `useLayoutEffect`?

Feature	`useEffect`	`useLayoutEffect`
When it runs	After paint	Before paint (after DOM mutation)
Use for	Async tasks, data fetching	Measuring DOM, sync layout

https://www.linkedin.com/in/sandeeppal

Blocking paint?	❌ No	✅ Yes (can cause jank)

🔍 Use `useLayoutEffect` only when layout measurement or synchronously modifying DOM is necessary.

---

26. How does `useRef` work and what are common use cases?

`useRef` creates a **mutable reference** that **persists across renders**.

✅ **Syntax:**

```
const ref = useRef(initialValue);
```

✅ **Use cases:**

- Accessing **DOM elements**
- **Persisting values** without causing re-renders
- Storing **previous values**

✅ **Example (DOM access):**

```
const inputRef = useRef();

function focusInput() {
  inputRef.current.focus();
}

return <input ref={inputRef} />;
```

✅ **Example (storing previous state):**

```
const prevCount = useRef();
useEffect(() => {
  prevCount.current = count;
```


https://www.linkedin.com/in/sandeeppal

```
});
```

---

27. Can you explain the `useContext` hook?

`useContext` lets you consume a **context value** without using a Context.Consumer.

✅ **Example:**

```
const ThemeContext = React.createContext("light");

function App() {
  return (
    <ThemeContext.Provider value="dark">
      <Toolbar />
    </ThemeContext.Provider>
  );
}

function Toolbar() {
  const theme = useContext(ThemeContext);
  return <div className={`theme-${theme}`}>Theme is {theme}</div>;
}
```

---

28. How do you create custom hooks?

Custom hooks are just **functions that use hooks**.

✅ **Example:**

```
import { useState, useEffect } from 'react';

function useWindowWidth() {
  const [width, setWidth] = useState(window.innerWidth);

  useEffect(() => {
```

https://www.linkedin.com/in/sandeeppal

```
    const handleResize = () => setWidth(window.innerWidth);
    window.addEventListener('resize', handleResize);
    return () => window.removeEventListener('resize', handleResize);
  }, []);

  return width;
}

// Usage
function Component() {
  const width = useWindowWidth();
  return <p>Window width: {width}</p>;
}
```

---

29. What rules should you follow when using hooks?

✅ **Rules of Hooks:**

1. **Only call hooks at the top level**
   (Don't call hooks inside loops, conditions, or nested functions.)

2. **Only call hooks from React functions**
   (functional components or custom hooks)

React enforces these rules with a linter: `eslint-plugin-react-hooks`.

---

30. How does `useMemo` optimize performance?

`useMemo` **memoizes a computed value** to avoid recalculating unless dependencies change.

✅ **Syntax:**

```
const memoizedValue = useMemo(() => computeExpensiveValue(a, b), [a, b]);
```


https://www.linkedin.com/in/sandeeppal

✅ **Use case:**
Avoid recalculating heavy logic on every render.

```
const filteredList = useMemo(() => {
  return items.filter(item => item.includes(searchTerm));
}, [items, searchTerm]);
```

---

31. What is `useCallback` and when should it be used?

`useCallback` **memoizes a function** to avoid unnecessary re-creations.

✅ **Syntax:**

```
const memoizedCallback = useCallback(() => {
  doSomething(a, b);
}, [a, b]);
```

✅ **Use case:**

- Prevents unnecessary re-renders of child components receiving functions as props.

```
const handleClick = useCallback(() => {
  console.log("Clicked!");
}, []);
```

---

32. How do hooks help avoid common pitfalls of class components?

Class Component Pitfall	Hook-Based Solution
`this` binding issues	✅ No `this` in hooks
Boilerplate code	✅ More concise with hooks

https://www.linkedin.com/in/sandeeppal

Sharing logic	✅ Custom hooks enable reuse
Complex lifecycle logic	✅ `useEffect` unifies side effects

✅ Hooks lead to **simpler, more readable, and reusable** code.

React State Management – Redux, Context API, and More

33. What is Redux and why might you use it with React?

Redux is a **predictable state container** for JavaScript apps, often used with React.

✅ **Why use Redux?**

- Centralizes app state
- Makes state **predictable and traceable**
- Useful in **large-scale apps** with **deeply nested** components
- Works well with **middleware** for async tasks (like API calls)

---

34. What are the core principles of Redux?

https://www.linkedin.com/in/sandeeppal

1. **Single source of truth**: All state is stored in a **single JavaScript object** (`store`).
2. **State is read-only**: You can't modify state directly — only via **actions**.
3. **Changes via pure functions**: **Reducers** are pure functions that take state and action, and return new state.

---

35. How do actions, reducers, and store work together in Redux?

🔁 **Data Flow in Redux:**

1. **Action**: A plain JS object that describes **what happened**.

```
{ type: 'INCREMENT' }
```

2. **Reducer**: A pure function that returns the **next state**.

```
function counterReducer(state = 0, action) {
  switch (action.type) {
    case 'INCREMENT':
      return state + 1;
    default:
      return state;
  }
}
```

3. **Store**: Holds the state and dispatches actions.

```
import { createStore } from 'redux';
const store = createStore(counterReducer);
```

---

https://www.linkedin.com/in/sandeeppal

36. What is the difference between local component state and global state?

Local State (`useState`)	Global State (Redux/Context)
Exists within a component	Shared across multiple components
Ideal for UI-level concerns	Ideal for app-wide data (e.g., auth, theme)
Cannot be accessed by siblings	Easily accessible across app

---

37. What is the Context API and how does it compare to Redux?

Context API is a built-in way to **pass data globally** without prop drilling.

Feature	Context API	Redux
Built-in?	✅ Yes	❌ No (external lib)
Use case	Small to medium apps	Large/complex apps
Boilerplate	Minimal	More (simplified by RTK)
Async support	Manual	Built-in via middleware
DevTools	❌ Limited	✅ Excellent support

---

38. How do you pass data using React Context?

✅ **Create context:**

```
const ThemeContext = React.createContext();
```

✅ **Provide context:**

```
<ThemeContext.Provider value="dark">
  <App />
</ThemeContext.Provider>
```


https://www.linkedin.com/in/sandeeppal

✅ **Consume context:**

```
const theme = useContext(ThemeContext);
```

---

39. What are middleware in Redux? Give examples.

Middleware are functions that sit between **dispatching an action** and **reaching the reducer**.

✅ Common uses:

- Handle async operations (API calls)
- Logging
- Error handling

✅ Examples:

- `redux-thunk`
- `redux-saga`
- `redux-logger`

✅ Example (redux-logger):

```
const logger = store => next => action => {
  console.log('dispatching', action);
  return next(action);
};
```

---

40. What are thunks and sagas?

https://www.linkedin.com/in/sandeeppal

Feature	Thunks (`redux-thunk`)	Sagas (`redux-saga`)
Concept	Functions returned from actions	Generator-based side effects
Complexity	Simple	Advanced
Syntax	Imperative (JS functions)	Declarative (generators/yield)
Use case	Basic async logic	Complex flows, retries, delays

✅ Thunk example:

```
const fetchUser = () => (dispatch) => {
  fetch("/api/user")
    .then(res => res.json())
    .then(data => dispatch({ type: "SET_USER", payload: data }));
};
```

---

41. How does Redux Toolkit simplify Redux usage?

Redux Toolkit (RTK) is the official, recommended way to write Redux logic.

✅ RTK Benefits:

- Less boilerplate
- Built-in `createSlice`, `createAsyncThunk`, `configureStore`
- Integrated DevTools
- Handles immutability under the hood

✅ Example:

```
const counterSlice = createSlice({
  name: 'counter',
  initialState: 0,
  reducers: {
```


https://www.linkedin.com/in/sandeeppal

```
    increment: (state) => state + 1,
  },
});

export const { increment } = counterSlice.actions;
```

---

42. How would you decide between Redux and Context API?

Scenario	Use
Simple data (theme, locale)	✅ Context API
Complex global state	✅ Redux
Needs middleware or DevTools	✅ Redux
Only few components need data	✅ Context
Large team/multiple features	✅ Redux Toolkit

🎯 Rule of thumb:

> Use **Context for simple static data**, and **Redux for dynamic or large-scale app state**.

---

43. How do you handle side effects in Redux?

✅ Using **middleware**, such as:

- **redux-thunk** – lets you dispatch functions
- **redux-saga** – handles complex side effects via generators
- **redux-observable** – uses RxJS

✅ Example with thunk:

https://www.linkedin.com/in/sandeeppal

```
const fetchData = () => async (dispatch) => {
  const response = await fetch('/data');
  const data = await response.json();
  dispatch({ type: 'SET_DATA', payload: data });
};
```

---

44. What are selectors in Redux?

Selectors are functions that **read and return data from the store**, often with memoization.

✅ Purpose:

- Centralize access to state shape
- Avoid duplication
- Optimize performance

✅ Basic selector:

```
const selectUser = (state) => state.user;
```

✅ With `reselect`:

```
import { createSelector } from 'reselect';

const selectCartItems = (state) => state.cart.items;

const selectTotalPrice = createSelector(
  [selectCartItems],
  (items) => items.reduce((sum, item) => sum + item.price, 0)
);
```

React Routing

https://www.linkedin.com/in/sandeeppal

45. What is React Router?

React Router is the **standard library** for routing in React applications. It enables navigation between different components or views without full page reloads, helping build **single-page applications (SPA)**.

- **Declarative routing**: Allows you to define routes using JSX.
- **Dynamic routing**: Routes can be dynamically rendered based on the state, props, or URL.

---

46. How do you implement routing in React?

You can implement routing in React using `React Router`. Here's how:

1. Install the necessary package:

```
npm install react-router-dom
```

2. Wrap your app with `BrowserRouter` (or `HashRouter`):

```
import { BrowserRouter as Router, Route, Switch } from
'react-router-dom';

function App() {
  return (
    <Router>
      <Switch>
        <Route exact path="/" component={Home} />
        <Route path="/about" component={About} />
      </Switch>
    </Router>
```


https://www.linkedin.com/in/sandeeppal

```
  );
}
```

3. Define routes using the `<Route />` component:

```
function Home() {
  return <h1>Home Page</h1>;
}

function About() {
  return <h1>About Page</h1>;
}
```

---

47. What is the difference between `BrowserRouter` and `HashRouter`?

Feature	`BrowserRouter`	`HashRouter`
URL structure	Clean URLs (e.g., `/home`, `/about`)	URLs include a # (e.g., `/home#about`)
Usage	Preferred for **modern apps** on a web server	Useful for apps without server-side routing (e.g., static sites)
History API support	Uses the HTML5 **history API**	Uses the URL's hash to manage routing
SEO friendliness	SEO-friendly (with server-side configuration)	Less SEO-friendly due to # in the URL

---

48. How do you create nested routes?

To create **nested routes**, place a `<Route />` inside another route's component.

Example:

https://www.linkedin.com/in/sandeeppal

```
import { BrowserRouter as Router, Route, Switch, Link } from
'react-router-dom';

function App() {
  return (
    <Router>
      <div>
        <nav>
          <Link to="/about/team">Team</Link>
          <Link to="/about/company">Company</Link>
        </nav>
        <Switch>
          <Route exact path="/about" component={About} />
          <Route path="/about/team" component={Team} />
          <Route path="/about/company" component={Company} />
        </Switch>
      </div>
    </Router>
  );
}

function About() {
  return <h2>About Page</h2>;
}

function Team() {
  return <h3>Team Page</h3>;
}

function Company() {
  return <h3>Company Page</h3>;
}
```

---

49. How do you pass parameters in routes?

https://www.linkedin.com/in/sandeeppal

You can pass **URL parameters** (also called dynamic routes) using `:` in the route path.

Example:

```
import { BrowserRouter as Router, Route, Switch, useParams } from
'react-router-dom';

function App() {
  return (
    <Router>
      <Switch>
        <Route path="/user/:id" component={User} />
      </Switch>
    </Router>
  );
}

function User() {
  const { id } = useParams(); // Access the dynamic parameter

  return <h1>User ID: {id}</h1>;
}
```

In the example above, visiting `/user/1` will display `User ID: 1`.

---

50. What is the difference between `Link` and `NavLink`?

Feature	`Link`	`NavLink`
Purpose	Navigate between pages	Navigation with active styling
Active style	No built-in active style	Automatically adds an active class when the route is active
Use case	Basic navigation	For creating navigation menus with active links (e.g., highlighting the active link)

https://www.linkedin.com/in/sandeeppal

Example of `NavLink`:

```
import { NavLink } from 'react-router-dom';

function Navbar() {
  return (
    <nav>
      <NavLink to="/" activeClassName="active">Home</NavLink>
      <NavLink to="/about" activeClassName="active">About</NavLink>
    </nav>
  );
}
```

---

51. How do you implement protected routes?

Protected routes are used to restrict access to certain routes based on some condition (e.g., user authentication). You can create a wrapper component that checks the condition and either redirects or renders the requested route.

Example:

```
import { Redirect, Route } from 'react-router-dom';

function ProtectedRoute({ component: Component, ...rest }) {
  const isAuthenticated = false; // Replace with your auth logic

  return (
    <Route
      {...rest}
      render={(props) =>
        isAuthenticated ? (
          <Component {...props} />
        ) : (
          <Redirect to="/login" />
        )
      }
    />
```


https://www.linkedin.com/in/sandeeppal

```
  );
}

function App() {
  return (
    <Router>
      <Switch>
        <Route path="/login" component={Login} />
        <ProtectedRoute path="/dashboard" component={Dashboard} />
      </Switch>
    </Router>
  );
}
```

In this example, if the user is not authenticated, they will be redirected to the login page when trying to access the `/dashboard` route.

---

52. How do you programmatically navigate in React Router?

You can navigate programmatically using the `useHistory` hook (React Router v5) or `useNavigate` hook (React Router v6).

For React Router v5 (`useHistory`):

```
import { useHistory } from 'react-router-dom';

function MyComponent() {
  const history = useHistory();

  const goToHome = () => {
    history.push('/');
  };

  return <button onClick={goToHome}>Go to Home</button>;
}
```

https://www.linkedin.com/in/sandeeppal

For React Router v6 (`useNavigate`):

```
import { useNavigate } from 'react-router-dom';

function MyComponent() {
  const navigate = useNavigate();

  const goToHome = () => {
    navigate('/');
  };

  return <button onClick={goToHome}>Go to Home</button>;
}
```

React Performance Optimization

53. What are common causes of performance issues in React apps?

Common causes of performance issues include:

1. **Unnecessary Re-renders**: React components re-render every time their state or props change. If not properly optimized, this can lead to **performance bottlenecks**.
    - **Example**: A large component tree where every small state change causes re-renders for the entire tree.
2. **Large Component Trees**: A deeply nested component structure can increase the time it takes for React to render.
3. **Heavy Computations in Render**: If you're performing heavy calculations directly in the render method, it can block the UI thread, resulting in slow rendering.
4. **Inefficient List Rendering**: Rendering large lists or complex components without optimization can degrade performance, especially if each item is re-rendered every time the parent changes.

https://www.linkedin.com/in/sandeeppal

5. **Excessive Reconciliation**: React's reconciliation algorithm can become slow if the state changes frequently or if the `shouldComponentUpdate` method isn't properly utilized.

6. **Event Handlers and Inline Functions**: Using inline functions inside `render()` or JSX can create new function references on every render, causing unnecessary re-renders.

---

54. How does `React.memo` improve performance?

`React.memo` is a **higher-order component** that memoizes functional components. It only re-renders the component if **its props have changed**.

- **Before**: Every re-render, even if props haven't changed.
- **After**: Prevents unnecessary re-renders when props remain the same.

```
const MyComponent = React.memo(function MyComponent({ name }) {
  return <div>{name}</div>;
});

// MyComponent will only re-render if the `name` prop changes
```

---

55. How do `useMemo` and `useCallback` help with performance?

useMemo:

`useMemo` is a hook that **memoizes** the result of an expensive function call and only recomputes the value when one of the dependencies changes.

- **Use case**: Expensive calculations or operations that don't need to be recalculated on every render.

Example:

https://www.linkedin.com/in/sandeeppal

```
const expensiveValue = useMemo(() => computeExpensiveValue(a, b),
[a, b]);
// expensiveValue will only recompute when `a` or `b` changes
```

useCallback:

`useCallback` is used to **memoize** a function so it doesn't get recreated on every render.

- **Use case**: Prevents function re-creation when passing functions down to child components (important when using `React.memo` or `PureComponent`).

Example:

```
const handleClick = useCallback(() => {
  // handle the click event
}, [dependencies]); // Recreate the function only when dependencies
change
```

---

56. What is code splitting and how do you implement it?

Code splitting is the practice of breaking up the bundle into smaller chunks so that only the required code is loaded when needed, improving **initial load performance**.

How to Implement:

1. Use **React.lazy** to dynamically import components.
2. Use **Suspense** to handle the loading state while the component is loading.

```
import React, { Suspense } from 'react';

const LazyComponent = React.lazy(() => import('./LazyComponent'));

function App() {
  return (
    <Suspense fallback={<div>Loading...</div>}>
```


https://www.linkedin.com/in/sandeeppal

```
      <LazyComponent />
    </Suspense>
  );
}
```

- **React.lazy** enables dynamic imports for code splitting.
- **Suspense** lets you specify a loading state while waiting for the component to load.

---

57. What is lazy loading in React?

Lazy loading is a design pattern where resources (like components) are loaded **only when they are needed** (e.g., when they enter the viewport or the user navigates to the route).

In React, lazy loading is typically implemented using `React.lazy` and `Suspense`.

Example:

```
import { Suspense } from 'react';
const HeavyComponent = React.lazy(() => import('./HeavyComponent'));

function App() {
  return (
    <Suspense fallback={<div>Loading...</div>}>
      <HeavyComponent />
    </Suspense>
  );
}
```

This ensures that the heavy component is only loaded when the app actually needs it.

---

58. How does React's `shouldComponentUpdate` method work?

https://www.linkedin.com/in/sandeeppal

`shouldComponentUpdate` is a lifecycle method in **class components** that determines whether a component should **re-render**. By default, a component re-renders when state or props change, but `shouldComponentUpdate` allows you to **optimize** this behavior.

- **Return `false`** to prevent a re-render.
- **Return `true`** (or omit the method) to allow a re-render.

Example:

```
class MyComponent extends React.Component {
  shouldComponentUpdate(nextProps, nextState) {
    // Prevent re-render if props haven't changed
    return nextProps.name !== this.props.name;
  }

  render() {
    return <div>{this.props.name}</div>;
  }
}
```

---

59. How can you optimize rendering lists in React?

Rendering large lists can cause performance issues if each item re-renders unnecessarily. Here are some strategies:

1. **Use `React.memo`** for list items that do not change often.
2. **Key Prop**: Always provide a **unique `key`** to each list item to help React efficiently update only the items that have changed.
3. **Virtualization**: Use libraries like **react-window** or **react-virtualized** to only render the items that are visible in the viewport.

Example of list optimization with `key`:

https://www.linkedin.com/in/sandeeppal

```
const List = ({ items }) => {
  return (
    <ul>
      {items.map(item => (
        <li key={item.id}>{item.name}</li>
      ))}
    </ul>
  );
};
```

---

60. What are pure components?

A **Pure Component** is a React class component that automatically implements `shouldComponentUpdate` with a shallow prop and state comparison.

- **Benefit**: It prevents unnecessary re-renders by performing a shallow comparison of `props` and `state`.

Example:

```
class MyComponent extends React.PureComponent {
  render() {
    return <div>{this.props.name}</div>;
  }
}
```

Pure components are a good choice when you know that your component only depends on `props` and `state` and you want to avoid unnecessary updates.

---

61. How does virtualization work in React lists?

Virtualization is a technique where only the items that are visible in the viewport are rendered, while the others are **not rendered** until they come into view. This significantly reduces the number of DOM nodes and boosts performance when rendering large lists.

https://www.linkedin.com/in/sandeeppal

Popular libraries for virtualization in React include:

- **react-window**
- **react-virtualized**

Example with `react-window`:

```
import { FixedSizeList as List } from 'react-window';

function MyList({ items }) {
  return (
    <List
      height={400}
      itemCount={items.length}
      itemSize={35}
      width={300}
    >
      {({ index, style }) => (
        <div style={style}>{items[index]}</div>
      )}
    </List>
  );
}
```

Only the visible items will be rendered, and as the user scrolls, new items will be loaded dynamically.

---

62. How do you debug performance issues in React apps?

Debugging performance issues can be done using several tools and strategies:

1. **React Developer Tools**:
    - Use the **Profiler** tab to measure the render times and identify slow components.

https://www.linkedin.com/in/sandeeppal

- Check for unnecessary re-renders and optimize with `React.memo` or `shouldComponentUpdate`.

2. **Console Logs and `performance.now()`**:
   - Log the time it takes for certain parts of your app to render and pinpoint slow areas.
3. **React's `useEffect` Dependency Analysis**:
   - If using hooks, make sure you are providing the correct dependencies to `useEffect` and `useMemo`, and avoid unnecessary recalculations.
4. **Third-party libraries**:
   - Use performance monitoring tools like **why-did-you-render** to detect unnecessary re-renders in functional components.

Example:

```
npm install @welldone-software/why-did-you-render
```

Testing React Components

63. How do you test React components?

You test React components by writing **unit tests** that simulate rendering, interaction, and lifecycle behavior.

Key Steps:

1. **Choose a Testing Library**: Use libraries like **Jest** (test runner) and **React Testing Library** (for testing React components).

https://www.linkedin.com/in/sandeeppal

2. **Write Tests**: Use functions like `render()` to simulate component rendering, and `fireEvent()` to simulate user interaction.

3. **Assertions**: Use `expect()` to assert the expected behavior.

Example:

```
import { render, screen, fireEvent } from '@testing-library/react';
import MyComponent from './MyComponent';

test('renders the component and interacts', () => {
  render(<MyComponent />);
  const button = screen.getByText('Click me');
  fireEvent.click(button);
  expect(screen.getByText('Clicked!')).toBeInTheDocument();
});
```

---

64. What is Jest and how is it used with React?

Jest is a **JavaScript testing framework** created by Facebook, commonly used for testing React applications. It comes with:

- **Test runner**: Executes test files.
- **Assertions**: Provides functions like `expect()` for asserting conditions.
- **Mocks**: Mocking capabilities for API calls or functions.
- **Snapshots**: Captures UI output at a given time.

How to use Jest with React:
Install Jest:

```
npm install --save-dev jest
```

1.

https://www.linkedin.com/in/sandeeppal

2. **Create tests**: Write tests in files with `.test.js` or `.spec.js` extensions.

3. **Run tests**: Jest automatically finds test files and executes them using `npm test` or `jest` command.

---

65. What is React Testing Library and how does it differ from Enzyme?

React Testing Library (RTL) and **Enzyme** are two popular testing utilities for React.

Feature	React Testing Library	Enzyme
Philosophy	Tests behavior from the user's perspective	Tests component internals and implementation
Test Focus	Interaction, rendering, and accessibility	Shallow rendering, component state, and props
Testing Approach	Encourages testing DOM behavior and accessibility	Encourages testing component internals and methods
Integration with React	Built with React's rendering behavior in mind	Requires enzyme adapter for different React versions
Recommendation	More modern, encourages better testing practices	Still used, but React Testing Library is more popular

Key difference:

- **React Testing Library** focuses on testing **the output of your components** (UI behavior), similar to how a user interacts with the app.
- **Enzyme** focuses more on testing **the internal implementation** (component state, methods).

---

66. How do you write a unit test for a React component?

https://www.linkedin.com/in/sandeeppal

A **unit test** checks a small unit of functionality, such as a React component. Here's how you can write a unit test:

1. **Render the component**: Use `render()` from React Testing Library.
2. **Simulate interactions**: Use `fireEvent()` or user-event for simulating actions like clicks, typing, etc.
3. **Assert behavior**: Use `expect()` to check the outcome, like whether an element is rendered or a function is called.

Example of a simple unit test:

```
import { render, screen, fireEvent } from '@testing-library/react';
import Counter from './Counter';

test('increments counter on button click', () => {
  render(<Counter />);

  const button = screen.getByText('Increment');
  fireEvent.click(button);

  expect(screen.getByText('1')).toBeInTheDocument();
});
```

In this test, you:

1. Render the `Counter` component.
2. Simulate a click on the "Increment" button.
3. Assert that the number `1` is now displayed.

---

67. How do you test component interactions and events?

https://www.linkedin.com/in/sandeeppal

To test **component interactions** like button clicks, form submissions, or other events, you can use `fireEvent` or **user-event** (for more realistic user interactions).

Example using `fireEvent`:

```
import { render, screen, fireEvent } from '@testing-library/react';
import MyComponent from './MyComponent';

test('button click changes text', () => {
  render(<MyComponent />);
  const button = screen.getByText('Click me');
  fireEvent.click(button);

  expect(screen.getByText('You clicked!')).toBeInTheDocument();
});
```

Example using user-event (better for simulating real user interactions):

```
npm install --save-dev @testing-library/user-event

import { render, screen } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import MyComponent from './MyComponent';

test('button click changes text', () => {
  render(<MyComponent />);
  const button = screen.getByText('Click me');
  userEvent.click(button);

  expect(screen.getByText('You clicked!')).toBeInTheDocument();
});
```

`user-event` is better because it simulates **real user actions** like clicks, typing, and more.

---

68. How do you mock API calls in tests?

To mock API calls in tests, you can use **jest.mock()** to mock functions or API calls.

https://www.linkedin.com/in/sandeeppal

Mocking `fetch` or `axios`:

Mock `fetch`:

```
global.fetch = jest.fn(() =>
  Promise.resolve({
    json: () => Promise.resolve({ data: 'mock data' }),
  })
);
```

1.

Mock `axios`:

```
import axios from 'axios';

jest.mock('axios');

axios.get.mockResolvedValue({ data: 'mock data' });
```

2.

Example of mocking `fetch` in a test:

```
import { render, screen, waitFor } from '@testing-library/react';
import MyComponent from './MyComponent';

test('fetches and displays data', async () => {
  global.fetch = jest.fn(() =>
    Promise.resolve({
      json: () => Promise.resolve({ message: 'Data loaded' }),
    })
  );

  render(<MyComponent />);

  await waitFor(() => screen.getByText('Data loaded'));

  expect(screen.getByText('Data loaded')).toBeInTheDocument();
});
```

https://www.linkedin.com/in/sandeeppal

Here, we **mock the `fetch` function** to return mock data, then test whether it appears in the component.

---

69. How do you test async behavior in React components?

Testing async behavior often involves waiting for a component to update based on **API calls** or **timeouts**.

Example:

- **Use `waitFor` or `findBy`** queries to wait for async changes.

```
import { render, screen, waitFor } from '@testing-library/react';
import MyComponent from './MyComponent';

test('loads data asynchronously', async () => {
  render(<MyComponent />);

  // Wait for the element that should appear after the async call
  await waitFor(() => screen.getByText('Loaded Data'));

  expect(screen.getByText('Loaded Data')).toBeInTheDocument();
});
```

In this case, `waitFor()` waits until the expected element appears in the DOM after an async operation completes.

---

70. What is snapshot testing?

Snapshot testing is a way of testing a component's rendered output by saving it to a snapshot file and comparing it with future renders. This helps detect any unintended changes in the UI.

https://www.linkedin.com/in/sandeeppal

How to write snapshot tests:
Run Jest with snapshot testing:

```
npm test -- --updateSnapshot
```

1.
2. **Write a snapshot test** using Jest:

```
import { render } from '@testing-library/react';
import MyComponent from './MyComponent';

test('matches snapshot', () => {
  const { asFragment } = render(<MyComponent />);
  expect(asFragment()).toMatchSnapshot();
});
```

- `asFragment()` captures the rendered output as a **DOM snapshot**.
- `toMatchSnapshot()` compares the output to a previously saved snapshot.

The first time the test is run, it saves the output to a snapshot file. On subsequent runs, it compares the output to the saved snapshot. If there are any changes, the test will fail.

https://www.linkedin.com/in/sandeeppal

Advanced React Concepts

71. What are Higher-Order Components (HOCs)?

A **Higher-Order Component** (HOC) is a **function** that takes a component and returns a **new component** with additional props or behavior.

- **Purpose**: HOCs are used for **code reuse**, **logic abstraction**, and enhancing components with common functionality (e.g., authentication, data fetching, etc.).

- **How it works**: HOCs are like decorators that wrap a component and add extra logic before rendering the component.

Example:

```
function withLoading(Component) {
  return function WithLoading(props) {
    if (props.isLoading) {
      return <div>Loading...</div>;
    }
    return <Component {...props} />;
  };
}

const MyComponent = ({ data }) => <div>{data}</div>;

const MyComponentWithLoading = withLoading(MyComponent);
```

In this example, `withLoading` is a higher-order component that adds loading state to `MyComponent`.

---

72. What is the render props pattern?

The **render props** pattern is a technique for sharing code between components using a prop whose value is a function (a “render prop”). This function can return JSX or other values, allowing more **dynamic behavior** and **custom rendering**.

- **Purpose**: It enables a component to expose its logic while letting its consumers define how the output is rendered.

Example:

```
class MouseTracker extends React.Component {
  state = { x: 0, y: 0 };

  handleMouseMove = (event) => {
    this.setState({
```

https://www.linkedin.com/in/sandeeppal

```
      x: event.clientX,
      y: event.clientY,
    });
  };

  render() {
    return (
      <div onMouseMove={this.handleMouseMove}>
        {this.props.render(this.state)}
      </div>
    );
  }
}

const App = () => (
  <MouseTracker render={({ x, y }) => (
    <h1>The mouse position is ({x}, {y})</h1>
  )} />
);
```

In this example, `MouseTracker` uses the **render prop** pattern to allow the parent component to decide how to render the mouse position data.

---

73. What are React Portals and when would you use them?

A **Portal** provides a way to render children into a **different part of the DOM** outside of the parent component's DOM hierarchy. This is particularly useful for scenarios like modals, tooltips, or popups that need to visually break out of their parent component but still maintain their React component state.

- **When to use**: When you need to render content outside the DOM hierarchy of a parent, without losing the React component context (e.g., for modals, overlays, etc.).

Example:

```
import React from 'react';
```


https://www.linkedin.com/in/sandeeppal

```
import ReactDOM from 'react-dom';

function Modal() {
  return ReactDOM.createPortal(
    <div className="modal">This is a modal</div>,
    document.getElementById('modal-root') // Renders outside the
normal DOM tree
  );
}

export default Modal;
```

In this case, the modal is rendered inside a specific part of the DOM (`modal-root`) even though it is a child of the `Modal` component.

---

74. What is reconciliation in React?

Reconciliation is the process by which React updates the **DOM** efficiently when a component's state or props change. React uses a **virtual DOM** to compare the new virtual DOM tree with the previous one and determines the minimal set of changes required to update the actual DOM.

Key Concepts:

- **Virtual DOM**: A lightweight representation of the real DOM.
- **Diffing algorithm**: React's algorithm compares the old and new virtual DOMs to find the differences (diffs) and apply the smallest set of changes.

Reconciliation Flow:

1. React creates a virtual DOM when the component is first rendered.
2. When state or props change, React creates a new virtual DOM.
3. The **diffing algorithm** compares the new virtual DOM with the old one.

https://www.linkedin.com/in/sandeeppal

4. The necessary updates are batched and applied to the real DOM.

---

75. What is React Fiber?

React Fiber is a complete rewrite of React's **reconciliation algorithm**. It is designed to improve the rendering performance and make React more **responsive** and capable of handling **asynchronous rendering**.

Key Features:

- **Incremental Rendering**: React Fiber allows rendering to be split into chunks, so it can pause and resume work, improving responsiveness.
- **Prioritization**: React can assign different priority levels to updates (e.g., user input vs. background updates).
- **Async Rendering**: Enables non-blocking UI updates, making React apps feel more responsive.

---

76. How does an error boundary work in React?

An **Error Boundary** is a React component that catches JavaScript errors in its child components, logs those errors, and displays a fallback UI. This prevents the entire app from crashing when an error occurs in a part of the component tree.

Usage:

1. Implement `componentDidCatch` or `static getDerivedStateFromError`.
2. Use a fallback UI to handle errors gracefully.

Example:

```
class ErrorBoundary extends React.Component {
  constructor(props) {
    super(props);
```

https://www.linkedin.com/in/sandeeppal

```
    this.state = { hasError: false };
  }

  static getDerivedStateFromError(error) {
    return { hasError: true }; // Update state to trigger a fallback
UI
  }

  componentDidCatch(error, info) {
    console.error("Error caught by Error Boundary:", error, info);
  }

  render() {
    if (this.state.hasError) {
      return <h1>Something went wrong!</h1>;
    }
    return this.props.children;
  }
}

export default ErrorBoundary;
```

You can wrap parts of your app in this **ErrorBoundary** to gracefully handle errors.

---

77. What are controlled side effects?

Controlled side effects refer to operations in React that happen as a result of state or props changes but are carefully managed, typically using **React Hooks** like `useEffect`.

- **Examples**: Fetching data, subscribing to an event, manually updating the DOM, etc.
- **Controlled**: The side effect is triggered in response to specific state changes and cleaned up appropriately.

Example of controlled side effect with `useEffect`:

https://www.linkedin.com/in/sandeeppal

```
import React, { useState, useEffect } from 'react';

function MyComponent() {
  const [data, setData] = useState(null);

  useEffect(() => {
    const fetchData = async () => {
      const result = await fetch('/api/data');
      const json = await result.json();
      setData(json);
    };

    fetchData();
  }, []); // Only runs once when the component is mounted

  return <div>{data ? JSON.stringify(data) : 'Loading...'}</div>;
}
```

Here, `useEffect` handles the **side effect** of fetching data when the component mounts, and the state is updated with the fetched data.

---

78. How do you implement server-side rendering (SSR) with React?

Server-Side Rendering (SSR) allows React components to be rendered **on the server** and the resulting HTML to be sent to the client. This improves the **initial loading performance** and helps with **SEO**.

SSR Process:

1. On the server, React renders the app to an HTML string using `ReactDOMServer.renderToString()`.

2. This HTML is sent to the client, where React "hydrates" it, attaching event listeners and making it interactive.

Example with `express` and React:

https://www.linkedin.com/in/sandeeppal

```
const express = require('express');
const React = require('react');
const ReactDOMServer = require('react-dom/server');
const App = require('./App');

const server = express();

server.get('*', (req, res) => {
  const content = ReactDOMServer.renderToString(<App />);
  res.send(`
    <html>
      <head><title>SSR Example</title></head>
      <body>
        <div id="root">${content}</div>
      </body>
    </html>
  `);
});

server.listen(3000, () => console.log('Server running on
http://localhost:3000'));
```

In this setup, React renders the app to HTML on the server, then the client "hydrates" it for interactivity.

---

79. What is static site generation (SSG) in React frameworks like Next.js?

Static Site Generation (SSG) is a technique where HTML pages are generated **at build time**. Unlike SSR, SSG pre-renders all pages during the build process, which results in **faster load times**.

Example with Next.js (SSG):

```
import React from 'react';

export async function getStaticProps() {
```


https://www.linkedin.com/in/sandeeppal

```
  const data = await fetchDataFromAPI();
  return { props: { data } };
}

function Page({ data }) {
  return <div>{data}</div>;
}

export default Page;
```

Here, `getStaticProps` is a Next.js function that fetches data during build time. The page is pre-rendered and served as static HTML.

---

80. What is hydration in React SSR?

Hydration refers to the process where React takes over the static HTML rendered by the server and **attaches event listeners** and **restores interactivity**. This happens once the JavaScript bundle is loaded on the client.

- **SSR**: Server renders the HTML.
- **Hydration**: React "hydrates" the server-rendered HTML to make it interactive.

This process allows the page to load quickly (thanks to the server-rendered HTML) and then become fully interactive once React takes over.

https://www.linkedin.com/in/sandeeppal

React Ecosystem & Tools

81. What is Next.js and why use it?

Next.js is a **React framework** that enables **server-side rendering (SSR)**, **static site generation (SSG)**, **API routes**, and more out-of-the-box. It simplifies React app development by providing a set of conventions and features that improve performance, SEO, and developer experience.

Why use Next.js?

- **Server-Side Rendering (SSR)**: Automatically generates HTML on the server for better SEO and faster initial load.
- **Static Site Generation (SSG)**: Pre-renders pages at build time, improving performance.
- **Automatic Code Splitting**: Only loads the necessary JavaScript for the current page.
- **File-Based Routing**: Routing is based on the file system, making navigation easier.
- **API Routes**: Allows you to build API endpoints directly inside your Next.js app.
- **Image Optimization**: Automatically optimizes images for better performance.

Example:

```
npx create-next-app my-next-app
```

In this setup, the app can be SSR or SSG-enabled, providing better performance and SEO out of the box.

---

82. What is Create React App?

Create React App (CRA) is a **boilerplate tool** to set up a modern React app without configuring build tools like Webpack, Babel, etc. It's designed to help you focus on writing code instead of spending time configuring tools.

Features:

- **Zero Configuration**: It sets up Webpack, Babel, ESLint, and other essential tools.

https://www.linkedin.com/in/sandeeppal

- **Development Server**: Provides a development server with hot reloading.
- **Production Build**: Automates production optimizations like minification, asset optimization, etc.

How to Use CRA:

```
npx create-react-app my-app
cd my-app
npm start
```

With CRA, you don't have to manually configure Webpack or Babel. It gives you everything you need to start building React applications right away.

---

83. What are common build tools used with React?

Some common build tools for React are:

1. **Webpack**: A module bundler that compiles JavaScript, CSS, images, and other assets. It's highly configurable and can be used for React apps, though tools like CRA automate this.
2. **Babel**: A JavaScript compiler that enables the use of modern JavaScript features (ES6+) and JSX syntax in React.
3. **ESLint**: A static code analysis tool that identifies problematic patterns in JavaScript, making sure your code follows a consistent style.
4. **Prettier**: A code formatter that enforces a consistent style across your codebase.
5. **Parcel**: A zero-config bundler that provides a fast alternative to Webpack.

Many of these tools are **integrated automatically** when you use CRA or Next.js, but they can be configured or customized in more advanced setups.

---

84. How do you handle CSS in React applications?

https://www.linkedin.com/in/sandeeppal

There are several ways to handle **CSS** in React applications, and each comes with its own pros and cons.

1. **Traditional CSS**:
   - You can import regular CSS files directly into your components.

```
import './App.css'; // Regular CSS file
```

2.
   - Use **className** to apply styles.
3. **CSS Modules**:
   - Scoped CSS that helps avoid name collisions.

```
import styles from './App.module.css';

function App() {
  return <div className={styles.container}>Hello, world!</div>;
}
```

4.
5. **Styled-components** (CSS-in-JS):
   - **CSS-in-JS** allows writing actual CSS inside JavaScript files.
   - Supports **dynamic styling** and **theming**.

```
import styled from 'styled-components';

const Button = styled.button`
  background: ${(props) => (props.primary ? 'blue' : 'gray')};
  color: white;
`;

function App() {
  return <Button primary={true}>Click Me</Button>;
```

https://www.linkedin.com/in/sandeeppal

```
}
```

6.
7. **Emotion**:
    - Similar to `styled-components`, but with additional features like **CSS prop** for inline styling.

```
/** @jsxImportSource @emotion/react */
import { css } from '@emotion/react';

const buttonStyle = css`
  background: blue;
  color: white;
`;

function App() {
  return <button css={buttonStyle}>Click Me</button>;
}
```

8.
9. **Tailwind CSS**:
    - Utility-first CSS framework that uses predefined classes for layout and styling. Great for rapid development.

---

85. What are CSS-in-JS libraries?

CSS-in-JS libraries allow you to write CSS styles **directly inside JavaScript files**, enabling better component encapsulation and dynamic styling. Popular libraries include:

1. **Styled-components**: Lets you create styled components that can be dynamically styled based on props or state.

2. **Emotion**: Similar to styled-components but with a focus on better performance and additional features.

https://www.linkedin.com/in/sandeeppal

3. **JSS**: A framework-agnostic CSS-in-JS solution that works well with React.

These libraries help solve problems like **global namespace conflicts** and **style collisions** in large React applications, allowing for **scoped, dynamic styles**.

---

86. What is PropTypes in React?

PropTypes is a **runtime type-checking** library used to validate the types of props passed to React components. It helps catch errors during development by ensuring that props match the expected data types.

Example:

```
import PropTypes from 'prop-types';

function MyComponent({ name, age }) {
  return <div>{name} is {age} years old</div>;
}

MyComponent.propTypes = {
  name: PropTypes.string.isRequired,
  age: PropTypes.number.isRequired,
};
```

In this example, `PropTypes.string.isRequired` ensures that the `name` prop is a string and is required. If it's not passed or is of the wrong type, React will display a warning in the console.

---

87. How do you do internationalization (i18n) in React?

Internationalization (i18n) in React can be handled using libraries that help manage translations, time formats, and currency formatting. The most common libraries are:

1. **react-i18next**: A powerful i18n framework for React, which integrates with the **i18next** library to manage translation strings and locale settings.

https://www.linkedin.com/in/sandeeppal

2. **FormatJS**: A set of libraries that help with formatting numbers, dates, and translations.

3. **react-intl**: A React wrapper around the **FormatJS** library, used for localization and internationalization.

Example with `react-i18next`:

```
npm install react-i18next i18next

import { useTranslation } from 'react-i18next';

function MyComponent() {
  const { t } = useTranslation();
  return <h1>{t('welcome_message')}</h1>;
}
```

In this case, `t('welcome_message')` will fetch the corresponding translation for the current language set in your app.

---

88. How do you handle authentication in React apps?

Authentication in React apps is typically done using **tokens** (JWTs) and **local storage** to persist user sessions.

Common Steps:

1. **Login**: When the user logs in, send a request to the backend (e.g., with **Axios** or **Fetch**) to authenticate and get a token.

2. **Store Token**: Store the authentication token in **localStorage** or **sessionStorage** for persistence across sessions.

3. **Private Routes**: Use a higher-order component or **React Router** to protect routes that require authentication.

4. **Logout**: Remove the token from storage and redirect to the login page.

https://www.linkedin.com/in/sandeeppal

Example:

```
function App() {
  const isAuthenticated = localStorage.getItem('authToken');

  return isAuthenticated ? <Dashboard /> : <Login />;
}
```

For handling **OAuth** or **third-party login** (e.g., with Google or Facebook), you might use a library like **Firebase** or **Auth0**.

---

89. What is React Native and how does it differ from React?

React Native is a framework for building **native mobile applications** (iOS and Android) using **React** and JavaScript. While **React** is for web development, **React Native** allows you to create mobile apps with a similar paradigm but using native components (e.g., `<View>` instead of `<div>`).

Key Differences:

- **React**: Builds for the web, rendering HTML and managing DOM.
- **React Native**: Builds for mobile, using native components and APIs for interaction with mobile devices (camera, geolocation, etc.).

Example in React Native:

```
import { Text, View } from 'react-native';

function App() {
  return (
    <View>
      <Text>Hello, React Native!</Text>
    </View>
  );
}
```

https://www.linkedin.com/in/sandeeppal

In React Native, the components map to **native UI elements**, such as `<Text>` for text, `<View>` for containers, and `<Button>` for buttons.

90. How do you manage forms in React with libraries like Formik or React Hook Form?

Managing forms in React can be tedious, but libraries like **Formik** and **React Hook Form** simplify form handling by managing state, validation, and submission.

Formik:

- Provides an easy way to manage form state, validation, and submission.
- Supports **validation** using schemas (e.g., **Yup**).

Example with Formik:

```
import { Formik, Field, Form } from 'formik';

function MyForm() {
  return (
    <Formik
      initialValues={{ name: '', email: '' }}
      onSubmit={(values) => console.log(values)}
    >
      <Form>
        <Field name="name" />
        <Field name="email" />
        <button type="submit">Submit</button>
      </Form>
    </Formik>
  );
}
```

React Hook Form:

https://www.linkedin.com/in/sandeeppal

- A lightweight alternative to Formik that uses React hooks to handle form state.
- It's more **performance-oriented** due to less re-rendering.

Example with React Hook Form:

```
import { useForm } from 'react-hook-form';

function MyForm() {
  const { register, handleSubmit } = useForm();

  const onSubmit = (data) => console.log(data);

  return (
    <form onSubmit={handleSubmit(onSubmit)}>
      <input {...register('name')} />
      <input {...register('email')} />
      <button type="submit">Submit</button>
    </form>
  );
}
```

Both libraries significantly reduce boilerplate and handle common issues like validation and form state management.

https://www.linkedin.com/in/sandeeppal

Hooks Deep Dive

91. How do you handle side effects with hooks?

Side effects are operations that occur outside of the React component's scope, such as:

- Fetching data from an API

- Setting up subscriptions
- Manually modifying the DOM

In React, side effects are handled using the `useEffect` hook.

Basic Example:

```
import React, { useState, useEffect } from 'react';

function Example() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    document.title = `You clicked ${count} times`;
  }, [count]); // Only run when `count` changes

  return (
    <div>
      <p>You clicked {count} times</p>
      <button onClick={() => setCount(count + 1)}>Click me</button>
    </div>
  );
}
```

`useEffect` allows you to perform side effects in function components. The second argument, the **dependency array**, specifies when to run the effect. If the array is empty, the effect runs only once after the initial render (like `componentDidMount`).

---

92. How do you avoid infinite loops with `useEffect`?

Infinite loops in `useEffect` occur when the effect continually triggers itself because the dependency array includes values that change as a result of the effect itself.

Common cause:

https://www.linkedin.com/in/sandeeppal

If a value inside `useEffect` is being updated, and it's in the dependency array, React will keep rerunning the effect.

How to avoid:

- **Use an empty dependency array** (`[]`) if the effect should run only once.
- **Be selective with dependencies** to avoid unnecessary re-renders.

Example:

```
useEffect(() => {
  // Perform API call or other side effects
  fetchData();
}, []);  // No dependencies, effect runs only once
```

If the dependency list has state values or props that change inside the effect, React will rerun it every time those values change. Be mindful of that!

---

93. How do you share logic between components with custom hooks?

Custom hooks are a powerful way to **share logic** between components in a reusable and encapsulated way. A custom hook is simply a JavaScript function that uses **React hooks** (like `useState`, `useEffect`, etc.).

Example of a custom hook:

```
// useLocalStorage.js
import { useState } from 'react';

function useLocalStorage(key, initialValue) {
  const [storedValue, setStoredValue] = useState(() => {
    const item = localStorage.getItem(key);
    return item ? JSON.parse(item) : initialValue;
  });

  const setValue = (value) => {
    setStoredValue(value);
```

https://www.linkedin.com/in/sandeeppal

```
    localStorage.setItem(key, JSON.stringify(value));
  };

  return [storedValue, setValue];
}

export default useLocalStorage;
```

You can then use this custom hook in any component:

```
import useLocalStorage from './useLocalStorage';

function App() {
  const [name, setName] = useLocalStorage('name', 'John');

  return (
    <div>
      <input
        value={name}
        onChange={(e) => setName(e.target.value)}
        placeholder="Enter your name"
      />
    </div>
  );
}
```

Custom hooks allow for **reusable logic** (like managing form state, fetching data, etc.) and encapsulation of behavior in a way that doesn't require repeated code.

---

94. How do hooks affect component lifecycle?

In **class components**, the lifecycle is managed using methods like `componentDidMount`, `componentDidUpdate`, and `componentWillUnmount`. With **hooks**, these are replaced with `useEffect`, which can mimic any lifecycle behavior.

https://www.linkedin.com/in/sandeeppal

Lifecycle Mapping:

- **componentDidMount**: Use `useEffect` with an empty dependency array (`[]`).
- **componentDidUpdate**: Use `useEffect` with dependencies.
- **componentWillUnmount**: Return a cleanup function inside `useEffect`.

Example:

```
useEffect(() => {
  // Equivalent to componentDidMount and componentDidUpdate
  console.log("Component mounted or updated");

  return () => {
    // Equivalent to componentWillUnmount
    console.log("Cleanup before component unmount");
  };
}, [dependencies]);  // Will run on mount and update based on
dependencies
```

---

95. What are some common pitfalls when using hooks?

1. **Incorrect dependencies in `useEffect`**: Forgetting to include all necessary dependencies can lead to bugs or infinite loops.
2. **Calling hooks conditionally**: Hooks should always be called at the **top level** of the component. Never call hooks inside loops, conditions, or nested functions.
3. **State updates with closures**: If using state in a function inside `useEffect`, make sure to account for the most recent state.
4. **Overusing `useEffect`**: Using `useEffect` for trivial things (e.g., directly manipulating the DOM when it's unnecessary) can lead to performance issues.
5. **State updates in loops**: Avoid unnecessary re-renders or state updates inside loops or effects that trigger multiple renders.

https://www.linkedin.com/in/sandeeppal

⚙️ Miscellaneous React Concepts

---

96. What are React keys and why are they important?

React keys help React identify which items in a list are changed, added, or removed. Keys provide a **stable identity** for each element, allowing React to optimize re-renders.

- **Important**: Keys should be **unique** for each sibling component.
- **Why important**: Without keys, React has to re-render all list items when the list changes, which can be inefficient.

Example:

```
const items = ['apple', 'banana', 'cherry'];

return (
  <ul>
    {items.map((item, index) => (
      <li key={index}>{item}</li>
    ))}
  </ul>
);
```

- **Tip**: It's better to use a **unique identifier** (e.g., `id`) as a key instead of using `index` if the list can change order or content dynamically.

---

97. How do you handle errors in React applications?

Handling errors in React is done using **Error Boundaries**. They are **React components** that catch JavaScript errors anywhere in their child component tree and log those errors or display a fallback UI.

https://www.linkedin.com/in/sandeeppal

Example of Error Boundary:

```
class ErrorBoundary extends React.Component {
  constructor(props) {
    super(props);
    this.state = { hasError: false };
  }

  static getDerivedStateFromError(error) {
    return { hasError: true };  // Update state to trigger fallback
UI
  }

  componentDidCatch(error, info) {
    console.error(error, info);
  }

  render() {
    if (this.state.hasError) {
      return <h1>Something went wrong!</h1>;
    }
    return this.props.children;
  }
}
```

You wrap your components inside **ErrorBoundary** to catch and handle errors gracefully.

---

98. How do you optimize React application for SEO?

To optimize React apps for **SEO** (Search Engine Optimization):

1. **Server-Side Rendering (SSR)**: Use frameworks like **Next.js** to pre-render content on the server. This allows search engines to crawl the content.

2. **Meta Tags**: Use libraries like **react-helmet** to dynamically set meta tags (`<title>`, `<meta>` tags) for SEO.

https://www.linkedin.com/in/sandeeppal

3. **Structured Data**: Use **JSON-LD** to provide structured data for search engines to understand the context of your content.

4. **Dynamic Content**: Preload important data to ensure that search engines can index key content.

5. **Ensure Accessibility**: Use semantic HTML to make content accessible to search engines.

Example (with `react-helmet`):

```
import { Helmet } from 'react-helmet';

function MyPage() {
  return (
    <div>
      <Helmet>
        <title>My SEO Optimized Page</title>
        <meta name="description" content="Description of my page for
SEO" />
      </Helmet>
      <h1>Welcome to My SEO Page</h1>
    </div>
  );
}
```

---

99. What is reconciliation and why is it important?

Reconciliation is the process React uses to **update the UI** efficiently when state or props change. React compares the **old virtual DOM** with the **new virtual DOM** and calculates the minimal set of changes required to update the real DOM. This makes React apps fast and efficient.

Why important:

- It helps React determine the minimal updates needed, avoiding unnecessary re-renders.

https://www.linkedin.com/in/sandeeppal

- React uses a **diffing algorithm** to compare previous and current states to optimize DOM updates.

---

100. How does React handle updates to the DOM?

React uses the **Virtual DOM** to handle updates. When state or props change, React creates a **new virtual DOM**, compares it to the previous version, and calculates the most efficient way to update the actual DOM.

Steps:

1. **State/Props Change**: Trigger a re-render.
2. **Create Virtual DOM**: A new virtual DOM tree is created.
3. **Diffing**: React compares the old and new virtual DOM trees.
4. **Reconcile**: React updates only the **necessary parts of the real DOM**, improving performance.

---

101. What is the difference between React and Angular or Vue?

React:

- **Library** for building UI, focusing on components and rendering.
- **Declarative and flexible**: Use JavaScript to write components.
- React has a rich ecosystem, but you often need additional libraries for routing, state management, etc.

Angular:

- **Framework**: Full-fledged framework for building web apps.

https://www.linkedin.com/in/sandeeppal

- Uses **TypeScript** by default and has built-in solutions for routing, HTTP requests, form handling, and more.
- **Two-way data binding**: Automatically synchronizes model and view.

Vue:

- **Framework**: Similar to React, but provides **two-way binding** and **template syntax**.
- Easier to integrate into existing projects.
- **Flexibility**: Offers the reactivity model and simplicity in syntax.

---

102. What is the role of JSX transpilation?

JSX transpilation refers to the process of converting JSX syntax (JavaScript XML) into regular JavaScript. Since browsers don't understand JSX directly, tools like **Babel** transpile JSX into valid JavaScript that browsers can execute.

Example:

```
const element = <h1>Hello, world!</h1>;
```

Babel transpiles the JSX into:

```
const element = React.createElement('h1', null, 'Hello, world!');
```

---

103. How do you implement drag and drop in React?

You can use libraries like **react-dnd** or **react-beautiful-dnd** for drag-and-drop functionality.

Example with `react-beautiful-dnd`:

```
npm install react-beautiful-dnd
```

```
import { DragDropContext, Droppable, Draggable } from
'react-beautiful-dnd';
```


https://www.linkedin.com/in/sandeeppal

```
function App() {
  const items = ['item1', 'item2', 'item3'];

  return (
    <DragDropContext onDragEnd={() => {}}>
      <Droppable droppableId="droppable">
        {(provided) => (
          <ul ref={provided.innerRef} {...provided.droppableProps}>
            {items.map((item, index) => (
              <Draggable key={item} draggableId={item}
index={index}>
                {(provided) => (
                  <li
                    ref={provided.innerRef}
                    {...provided.draggableProps}
                    {...provided.dragHandleProps}
                  >
                    {item}
                  </li>
                )}
              </Draggable>
            ))}
            {provided.placeholder}
          </ul>
        )}
      </Droppable>
    </DragDropContext>
  );
}
```

---

104. What is concurrent mode in React?

Concurrent Mode is an experimental feature in React that enables the rendering process to be **interruptible**. It allows React to work on multiple tasks at once, making apps feel more

https://www.linkedin.com/in/sandeeppal

responsive by prioritizing higher-priority updates (like animations or user input) over lower-priority ones.

---

105. How do Suspense and React.lazy work?

React.lazy enables you to dynamically import components only when they are needed, enabling **code splitting** and **lazy loading**.

```
const LazyComponent = React.lazy(() => import('./LazyComponent'));

function App() {
  return (
    <React.Suspense fallback={<div>Loading...</div>}>
      <LazyComponent />
    </React.Suspense>
  );
}
```

Suspense is a wrapper around lazy-loaded components that shows a **loading fallback** while the component is being loaded.

https://www.linkedin.com/in/sandeeppal